SAMS Kernel

Why the kernel exists: rationale, architecture and use cases

Version 1.28.1 · 28 October 2026
For a TI-99/4A with an F18A video card and a SAMS memory card
MIT License, © 2026 Rebecca Sutter

Contents

1. The idea in one paragraph

2. The problem: what a program must solve before it does anything of its own

3. Where it came from

4. Goals and non-goals

5. The shape of the system

6. Design decisions, and the alternatives they were chosen over

7. Use cases

8. What it is not for

9. Constraints that shaped the design, and what each cost

10. Deciding whether to build on it

11. Where each part of the documentation is

This document explains **why** the kernel is the way it is, **what it is for**, and **how its parts fit together**; it does not teach you to use it. It is for someone deciding whether to build on it, someone changing it, and someone who wants the reasoning behind a decision that looks odd. For the mechanics read ARCHITECTURE.md (what happens between power-on and the first frame, the memory maps); for writing a program read PROGRAMMERS-GUIDE.md.

A note on honesty that applies to every section: **the kernel as packaged has been run in a simulator and not on a real TI-99/4A.** Its parts were proven on real hardware inside the programs it was taken from (Press and The Lantern Coast); what is new since then (`GOMOD`, the page pool, the file service's handles, the disk start, the workbench) has not been. Where this document says "works" it means "works in the simulator, with TI's own console and disk-controller ROMs"; HARDWARE-CHECKLIST.md lists what is still to be tried and how.

1. The idea in one paragraph

A TI-99/4A program that wants a big, colourful text screen, a keyboard that behaves, more than 64K of memory, files on disk and sound has to solve the same dozen low-level problems before it does anything of its own, and the machine makes each of them awkward. The kernel solves them **once**, as a small resident layer (6.9K of an 8K bank), and gives the program a simple shape to fill in: *a module that is called once a frame with the key that was typed*. Everything the program does is written in its own modules; everything the machine makes difficult is the kernel's. It is delivered as a cartridge image or as a program on a disk, and it can be built, edited and assembled on the TI itself.

2. The problem: what a program must solve before it does anything of its own

Each row is a fact about the machine and what it costs a program that ignores it.

The machine	What it costs	What the kernel does
The CPU addresses **64K**; the only fast RAM is **256 bytes** of scratchpad; the 32K expansion RAM is 8 bits wide with wait states	A program that grows past 32K must manage memory by hand; scratchpad is scarce and shared with the console	Puts the kernel and the program in **SAMS** pages, keeps **windows** fixed or mapped on request, gives the program about 7K of data RAM (`APPRAM`) and a software stack in RAM
A **SAMS** card holds up to 16 MB in 4K pages seen through the CPU's windows	Every page switch is an I/O write that must not be interrupted; a window remapped under running code crashes it	`PAGEB`, `MAPE`/`MAPF`/`MAPD`, a **page pool** (`PALLOC`/`PRELS`), `MODCALL`/`GOMOD`, and the rule that code never remaps the window it runs in
A cartridge ROM is **8K** at a time, switched by bank	Code that is switched out from under itself, and no room for a real program	The **loader copies every bank into SAMS at start-up**, once; after that nothing is banked
The VDP is behind **two ports**; an **interrupt** between the two bytes of an address write corrupts the address	Hard-to-find screen corruption	Interrupts are masked everywhere except inside the frame wait (`VSYNC`); the main loop is built round it
The stock VDP is 40 columns, 16 colours, one colour per 8 cells	A text program looks like 1981	Requires an **F18A** (or PICO9918): **80 x 30 text, a colour per cell**, every F18A mode through `VMODE`, a colour *scheme* of named roles
The keyboard is a matrix: shift, function and control are keys, ALPHA LOCK is a switch, joysticks share the matrix	Each program writes its own scan and gets repeat, lock and chords wrong	One decoded key stream (`KPOLL`, `KEYGET`, `KWAIT`, `KFLUSH`), repeat, lock, joystick-as-keys, `FCTN` combinations as named codes
Files are reached by **device service routines** in cards' ROMs, found by scanning CRU addresses, with PABs in VRAM and different controllers behaving differently	A page of fragile code per program; works with one controller	`FOPEN`/`FREAD`/`FWRITE`/`FCLOSE`/`FLOAD`/`FSAVE`, three file handles, and the DSR lookup and power-up, tested with the real ROMs of **TI's, the CorComp, the Myarc DDCC-1 and the Myarc HFDC** controllers and a model of a TIPI
The sound chip is written a byte at a time on a clock	A song is a timing problem	A **music player** in the frame loop, a compact song format, a compiler from MIDI, text and TXA's `.txm` notation
A bare TI cannot tell you what is wrong with an F18A or a SAMS card	A program that assumes both crashes on a machine that lacks one	The **loader checks the machine** (an F18A answers, a mapper answers, enough pages, every page holds what is written to it) and shows a **refusal screen** that says

		what is missing
The console's own Quit leaves the machine in whatever state you left it	The next program sees your mess	A clean power-up state and a **clean exit**

None of these is hard to solve once. Every one is hard to solve *again* in each program, and each is a place where a program fails in a way that has nothing to do with what it does.

3. Where it came from

The kernel is not a design made in the abstract. It is **Press 2.0's resident kernel, itself descended from The Lantern Coast's**, with everything specific to a word processor taken out (`docs/FROM-PRESS.md` lists what came over, what was renamed and what was left behind).

	What it was	What it taught
The Lantern Coast	an Ultima-style role-playing game in TMS9900 assembly for an F18A and SAMS	scenes as banked code in one window; the F18A text screen; the first loader and pager
Press 2.0	a word processor on the same platform	scenes became **modules**; a keyboard layer that a real typist can use; the page-table-and-window approach to a document larger than memory; files through DSRs
The SAMS Kernel	what both needed, with nothing of either	generalised: table-driven loader, a page pool, module switching, `VMODE`, a workbench, then (1.17 to 1.27) the features that the engine and interpreter projects asked for

The design principle that came from this lineage is the rule the generalisation followed: **everything an application needs, nothing that only one application needs.** A routine is in the kernel only if at least two different programs would otherwise carry their own copy.

Programs the kernel was generalised from or for: The Lantern Coast, Press 2.0, the dBase-style database F18Base (built on Press's platform), the text-adventure engine TXA (its engine and workbench carry their own versions of the same layers: a pager, text, keys, VDP and F18A code), and the planned Python-subset interpreter TIthon. **None of them has been rebased on the packaged kernel yet** (`docs/FROM-PRESS.md` says what rebasing Press would involve; Press itself has not been changed). What *has* happened is that TXA and TIthon each sent a list of what they would need, and the kernel grew to meet it: file load and save (`FLOAD`, `FSAVE`), a frame hook and a break flag, single-page modules and window `>D000`, a float-package interface (`FPCALL`), common code at `>D000`, code that lives in application RAM, a cycle-counting model, the disk start and `PRUN`, and TXA's music notation as the default. TXA-proposal-assessment.md and TIthon-proposal-assessment.md say what was taken, what was changed and what was declined.

The versions in one table

Version	What it added
1.0 - 1.9	the first releases: the kernel as a package (loader, pager, video, text, keyboard, music, files, modules, the pool), the simulator and its tests, the programmer's guide, the workbench with its native assembler
1.10 - 1.16	the CorComp and Myarc DDCC-1 controllers with their real ROMs and double-density disks; the Myarc HFDC; `LINEIN`; resident code in the kernel's pages
1.17	the standalone editor example, the sources disk, the scrolling input field
1.18 - 1.19	`"omit"`: leave out music, files, the pool, the graphics modes: the kernel shrinks from 6.9K to about 3.6K
1.20 - 1.22	`FLOAD`/`FSAVE`, the `.mape`/`.mapf`/`.mapd` macros, the frame hook and break flag, a faster pool, single-page modules
1.23 - 1.24	the float package (`FPCALL`), common code at `>D000`, the cycle model
1.25	**the disk start and `PRUN`**: a kernel program can be a disk
1.26	code in application RAM (`"appram_code"`)
1.27	`.txm`, TXA's notation, as the default music notation

4. Goals and non-goals

Goals

1. A program author writes **only what is their own**, in a small module with three entry points.
2. **It fails loudly and early** on a machine it cannot run on, with a screen that says why.
3. **The same source** builds on a PC and assembles on the TI itself.
4. **Everything claimed is tested**, with the real ROMs where there are any, and **what is not tested says so**.
5. **A small kernel**: every optional service can be left out; the kernel is a few thousand bytes, not a system.
6. **No surprises at the hardware**: every register a routine changes is in its header, every hazard that cost a day is in a checklist.

Non-goals

- **Not an operating system.** There is no scheduler, no processes, no memory protection. A program owns the machine; the kernel is a library with a main loop.
- **Not for an unexpanded console.** An F18A and 512K of SAMS are required; there is no 40-column, 16K fall-back.
- **Not a graphics engine.** `VMODE` reaches every F18A mode, but the kernel *draws for you* only in the glyph modes (Text 80/40, Graphics I). Sprites, Graphics II, Multicolor, the layers and the bitmap you write to VRAM yourself (`docs/VIDEO-MODES.md` shows what each takes).
- **Not a replacement for GPL, TI BASIC or Extended BASIC programs**, and it does not run them.
- **Not a language.** It is assembly. A language built on it (TIthon) is a program on top.
- **Not portable to other machines.** The point is to use this machine's hardware well.

5. The shape of the system

```
power on
  |
  v
[ the console's menu ]  ->  a cartridge (FinalGROM, bank 0) or Editor/Assembler option 5 (a disk's bootstrap)
  |
  v
[ loader ]   checks the machine (F18A, SAMS size, every page), then copies the banks into SAMS
  |          (from the cartridge's banks, or read from the disk's bank files)          refusal screen on any failure
  v
[ kernel, SAMS pages >0A-0B, >A000-BFFF ]  start-up, then forever:  wait for the frame; poll the keys; step the music;
  |                                        call the running module's TICK with the key
  |   services, called by modules:  video/text | keys | pager + page pool | modules | files | music | float | time
  v
[ the application's modules, slot C, >C000 ]   INIT once, TICK each frame, REDRAW when the screen is lost
```

The memory it manages

Window	Address	What is there
scratchpad	`>8300-83FF`	the kernel's hot variables and workspace (the console's and the DSRs' words are respected)
slot A	`>2000-3FFF` (pages `>02-03`, **never remapped**)	the kernel's variables, the software stack at the top, **`APPRAM`**: about 7K for the application
the kernel	`>A000-BFFF` (pages `>0A-0B`, never remapped)	the kernel's code, 6.9K of 7.5K it may use (the page-pool bitmap takes the end of the bank)
slot C	`>C000-DFFF`	**the running module**: 8K, or 4K in the single-page form; remapped only by `PAGEC`, called only from the main loop and `MODCALL`
slot B	`>E000-FFFF`	two 4K **windows**, E and F, which a program points at any page with `MAPE`/`MAPF`
SAMS	pages `>10` upward	the data pages, the module pages, the **page pool**
VRAM	16K, as the console sees it	the screen, the pattern table, the colours, the file service's buffers at `>3600-37D7`, the disk controller's above them

The main loop in one paragraph

Every frame the kernel waits for the vertical blank (`VSYNC`: interrupts are allowed here, and only here; the frame hook and the music step with it), scans the keyboard (`KPOLL`), takes the next decoded key (`KEYGET`) and calls the running module's `TICK` with the key in R0 (0 if none); then it checks whether the program asked to quit or to switch modules. **A module never has a main loop of its own**: it does a little work in `TICK` and returns, which is what lets the kernel poll the

keyboard, count frames and play music between calls. A module that must work longer calls `KIDLE`, which keeps the music and keyboard going while it does.

What a module is

Up to 8K (or 4K) of code at `>C000`, starting with a table of entry points (`INIT`, `TICK`, `REDRAW`, one reserved, then your own). It cannot call another module's routines, because they are not mapped; it uses `MODCALL` (map in, call, map back) or `GOMOD` (switch). Modules share state only through addresses in slot A that they agree on in a shared file. The constraint looks heavy and is the point: it forces a program into pieces that fit, and makes crossing a boundary cheap and explicit.

6. Design decisions, and the alternatives they were chosen over

The loader copies every bank into SAMS, once, at start-up. The alternative, switching cartridge banks while running, means the code you are executing can disappear from under you whenever a bank is switched, and every module call is a bank switch. Copying costs a pause at the start (the first screen appears after about 84,000 instructions for the small examples, more for a big program; real time is not measured) and buys a flat system in which modules are just pages. It also makes the disk start nearly free: the loader's only change is where the banks come from.

The kernel and the program are in fixed SAMS pages; modules go in one window. Press began with scene banks in a window; modules are the same idea made general. 8K is tight, and that is useful: it forces structure. `MODCALL` and `GOMOD` make crossing it cheap. The single-page (4K) form and the common-code window `>D000` exist because programs that were *almost* one module found a second 8K excessive.

A software stack in slot A, not scratchpad. The scratchpad is 256 bytes shared with the console, and the console's interrupt and the DSRs use it too. A 256-byte stack in slot A costs nothing the program needs.

Interrupts are masked except in `VSYNC`. A TMS9900 interrupt between the two bytes of a VDP address write corrupts the address, and the symptom is a glyph in the wrong place once an hour. One place allows the interrupt, and everything else runs with `LIMI 0`.

`GOMOD` is deferred, `MODCALL` is immediate. A module that asks to switch is running in slot C; remapping slot C under it would pull its code away mid-routine. So `GOMOD` only records the request, and the main loop does the mapping when the caller has returned. `MODCALL` maps the target in, calls it and maps the caller back, returning to it.

One calling convention, written down. R0-R9 are free, each routine's header lists every register it can change *including through what it calls*, R10 is the stack, R11 is return, R12 is CRU, R13-R15 are untouched (context switch and GPL workspace). Results are in registers, never in flags from a routine that ends with `.leave`. A convention without enforcement drifts; this one is checked: `tools/docindex.py --check` fails if a public routine has no header, and the headers generate `docs/ROUTINES.md`. *A real example of what the convention is for:* while writing the workbench manual, the screen's error list was found to show the same message for every error. It had read the error code from a register after calling the number printer, whose header says it uses R0-R8. The fix was one line; the cause is exactly the failure the header exists to prevent, and it had passed every test because the tests checked the error table, not the screen.

A page pool, not static page assignment. An application with data of unknown size (an editor's file, a database's index) cannot assign pages at build time. `PALLOC` hands out pages from a bitmap and `PRELS` returns them. The pool's bookkeeping was made fast (a byte scan, and `PALLOCA`) when the interpreter project asked for thousands of pages: with 3,900 in use the first version took about 107,000 instructions to find the next, the new one 2,589.

The file service is three handles over the DSR, not an operating-system file API. The controller finds an open file by its PAB's VRAM address, so each handle has its own PAB; the record buffer is shared because every call finishes with it; all of it lies below the controller's own buffers. Making this work with the real ROMs of four controller families (and a model of a fifth) cost far more than anything else in the kernel, and it is the part a program author should never have to see.

Colour is by role, not by number. A scheme maps named roles (text, bar, bold, mark, error, selection) to 16 palette words, so changing the scheme is 32 bytes of I/O and programs that use roles follow it.

Optional parts can be left out. `"omit"` removes music, files, the pool, the graphics modes and enhanced colour, taking the kernel from 6.9K to about 3.6K for a program that needs none of them. Three further levers (`"resident"`, `"common"`, `"appram_code"`) put shared code in the kernel's pages, in a shared 4K window, or in application RAM; `docs/RESIDENT.md` says what each costs and why the options are what they are.

A build driven by one JSON file. `app.json` names the modules, data, music, options; `skbuild.py` writes the generated configuration, assembles everything, filters the kernel's symbols down to the public ones (`kernel.equ`) and writes the

cartridge image (or the disk). A module may not reuse a name the kernel exports. The kernel's internals stay free to change.

The simulator is the test bench, with the real ROMs. Almost nothing about this machine can be learned from documentation alone: the sound chip's timing, a DSR's use of scratchpad, a disk controller's retry behaviour. So the project carries a TMS9900 simulator with the F18A, SAMS, a keyboard, a model of the sound chip, and the real ROMs of TI's console and disk controller and of the other controllers (TI's are not in the repository: the tests use them when you supply them), and every behaviour is checked by key scripts and by comparing bytes. Two things follow. First, a check is only as good as the model: the disk-controller model once accumulated a seek's target bytes and, at 256 of them, overwrote a sector of the disk image; it was found only because a longer scenario reached 256. Second, **the claims are labelled**: what the simulator checks, what a real TI has shown, and what is neither (the checklist).

A workbench that runs on the TI. An editor, an assembler and a launcher, written on the kernel, so that a program can be written without a PC. The assembler is byte-identical to xas99's output on every example module and on probes that use every instruction in every operand form. Its limits (one module of 4K, no macros) are what fit in the memory it has; they are stated in its manual. It is also the kernel's largest example of a program whose data does not fit in the memory it can see (the editor, `docs/PROGRAMMERS-GUIDE.md` appendix C).

The disk start and `PRUN`. A kernel program can be a disk: Editor/Assembler option 5 loads a small bootstrap that checks the machine as the cartridge loader does, reads the program's banks from the disk and starts the kernel. That serves anyone without a FinalGROM, and it serves a program that builds programs: `PRUN` replaces the running program with another disk program, which is what "run the game" in a workbench needs. It was done by changing the loader's *source of banks*, not its logic, so the checks and refusal screens are the same code (`docs/DISKSTART.md`).

7. Use cases

What each kind of program needs, and what in the kernel serves it. The column "Where it bites" is the limit a program of that kind meets first.

Use case	What it needs	What serves it	Where it bites
A text-heavy tool (editor, word processor, workbench)	a big screen, a real keyboard, a document larger than RAM, files	Text 80 x 30 with roles; `KPOLL` chords; the page pool and windows (the editor's line table); the file service; `LINEIN`	7K of data RAM: the document must live in pages; 80 columns per line in the editor's format
A game made of scenes (a role-playing game, an adventure)	many screens of code and data, music, a fast frame	modules per scene; data pages; the music player; the frame hook; `VMODE` and the tile layer	8K per module; sprites and tiles are drawn by the program
An engine or interpreter (a text-adventure engine, a language)	a large program, a large data set, arithmetic, files, sound	single-page modules and common code; `"appram_code"` for the dispatch loop; the pool; `FPCALL` for floating point; `FLOAD/FSAVE`; the cycle model to budget a loop; `.txm` music	the data set in SAMS must be mapped a page at a time; the float package's fixed scratchpad words
A database (a dBase-like tool)	records on disk, an index larger than RAM, forms	the file service (three handles); the pool for index pages; `LINEIN` for fields; Text 80	disk speed; three open files
A music or sound program	a player, songs in a notation, effects	`MUPLAY`, SKM, `skmusic.py` (MML, MIDI, `.txm`); `SNDTONE`	one song at a time; **effects that borrow voices from the song (`SFXPLAY`) are not built**
A diagnostic or utility (a key tester, a disk browser, a display-mode tour)	the machine's facts, without writing a loader	the loader's checks, `VMODE`, the file service's catalog, `examples/diag`, `examples/browser`, `examples/modes`	the F18A and SAMS are required
A program that launches programs	start another program, from a disk	the disk start and `PRUN`; the workbench's `R`	one program at a time; no state passed except through files
Learning	a small, readable complete program; a way to try changes	`examples/hello` (the smallest), the workbench on the TI, the annotated examples, the programmer's guide	the guide is long; the examples are commented to be read

Case notes

The editor (and the workbench). The text of a file lives in pool pages, 51 lines of 80 bytes to a page, in the order of a table of slot numbers, so inserting a line moves a table and never the text. The editor is 2.7K of code holding up to 2,040 lines with 7K of RAM. The same pattern (a table that says where things are, a window that is pointed at the thing you need) is how a database holds an index and how an interpreter holds a heap. It is the worked example of the kernel's central idea.

An interpreter. An interpreter spends its life in a dispatch loop. Put the loop in application RAM (`"appram_code"`), where it is always mapped and never swapped; keep the program it interprets in pages; call the float package through `FPCALL` (which runs a console ROM routine in the GPL workspace and reads its status); let the pool hand out heap pages. The cycle model (`tools/cycles.py`) counts a loop's cost in real clock cycles, with the wait states of expansion RAM, so a design that looks fast in instruction counts can be seen to be twice as slow in memory that has wait states (the same `DEC`/`JNE` loop: 0.437 s in the scratchpad and 0.786 s in expansion RAM).

A game with music. Compile the tunes at build time (`.txm` is the default notation; MML and MIDI also work) into pages; `MUPLAY` starts one and the frame loop steps it. The player writes the same bytes to the chip that the compiler's model says it should, frame by frame, and a test compares them.

A program that builds programs. The workbench assembles a source on the TI and runs it as a module; `PRUN` runs a whole disk program. Together they make a development environment that needs nothing but the machine, a disk and (for the bootstrap files) a PC once.

8. What it is not for

- A program that wants **fast action graphics**: sprites, scrolling, Graphics II tile engines are reachable, but you write them; the kernel's strength is text, memory and structure.
- A program for a **machine without an F18A or SAMS**: it will not start, and says why.
- A program that needs **more than one running thread of its own**: there is one `TICK`; the frame hook is a hook, not a scheduler.
- A program that must be a **ROM cartridge on a real, unmodified console with no SAMS**: use a conventional design.
- A program that wants **GPL** or **TI BASIC**'s environment.

9. Constraints that shaped the design, and what each cost

Constraint	Consequence in the design
About 7K of data RAM	the page pool, windows, the editor's table; code that keeps its own state small
Jumps reach +-127 words	the assembler reports `jump out of range`; the style is short routines and `B @label` for far jumps
One calling convention for everything	every routine's header lists its registers; a bug like the workbench's error list is caught by reading, not by testing
DSR state lives in VRAM and the scratchpad	the file service powers the cards up itself and keeps its buffers below the controller's own (`>3600-37D7`, under `>37D8`); the disk start's bootstrap hands over a clean machine so that it does so again
The sound chip's timing	the player runs in the frame loop and writes exactly the bytes the compiler decided
A real TI's behaviour is not in the manuals	the simulator uses the real ROMs; the claims are labelled; the checklist lists what must be seen on a real machine

10. Deciding whether to build on it

Build on the kernel if the program is **assembly for an F18A and SAMS machine**, is **text- or structure-heavy**, and would otherwise carry its own loader, pager, keyboard and file code. It saves a month and a class of bugs; it costs 6.9K (down to 3.6K) of the 7.5K it may use, a module shape to fit, and the discipline of the calling convention.

Do not build on it if the program is a small single-purpose cartridge, needs sprites and speed rather than text and

memory, or must run on a console without the cards.

If you do: read `docs/PROGRAMMERS-GUIDE.md` part 0 (what you get, what you need), then part 1 (Hello, kernel); try WORKBENCH-MANUAL.md's first tutorial on the TI or the simulator; and read part 14 of the guide, the checklist of what has already cost a day.

11. Where each part of the documentation is

You want	Read
to write a program	PROGRAMMERS-GUIDE.md; every routine in ROUTINES.md
to use the workbench	WORKBENCH-MANUAL.md; how it is built: WORKBENCH.md
to know what happens inside	ARCHITECTURE.md, HARDWARE.md, CONVENTIONS.md
to start a program from disk	DISKSTART.md
music	MUSIC.md
every display mode	VIDEO-MODES.md
shared code: where to put it	RESIDENT.md
to try it on a real TI	DEVELOPING-ON-A-TI.md, HARDWARE-CHECKLIST.md
where it came from	FROM-PRESS.md
what is next	ROADMAP.md